M2S-95 LAB: Terraform for Managed-to-SaaS Migration

> **Series:** M2S — Managed to SaaS Migration | **Reference:** 95 — Terraform Migration LAB | **Created:** July 2026 | **Last Updated:** 07/24/2026

An appendix LAB for teams migrating Dynatrace Managed to SaaS with the Terraform provider instead of — or alongside — the SaaS Upgrade Assistant.

This is **not** a Terraform tutorial. Provider installation, authentication models, state backends, module design, and GitOps promotion are covered in the AUTOM series and are not repeated here. What this LAB adds is the part AUTOM does not cover: the Managed-source-specific mechanics — what a Managed cluster can and cannot hand you as HCL, how to keep entity IDs from breaking your configuration on arrival, and where each Terraform action belongs in the nine-step M2S journey.

> **M2S Migration Journey — 3 Phases / 9 Steps**
>
> **Plan:** 1. Discover | 2. Strategize | 3. Design
>
> **Upgrade:** 4. Prepare | 5. Execute | 6. Integrate
>
> **Run:** 7. Enable | 8. Expand | 9. Optimize
>
> This LAB is an **appendix**, not a tenth step. It overlays the existing steps — see section 7.

Table of Contents

Prerequisites

Requirement	Detail
Terraform CLI	1.0+ with the `dynatrace-oss/dynatrace` provider on your `PATH`
Provider familiarity	AUTOM-04 (provider configuration, auth model, state) and AUTOM-09 (repo layout, state backend, promotion) — this LAB assumes both
Managed source access	Cluster admin plus an environment API token on the source Managed environment
SaaS target access	A provisioned SaaS tenant (M2S-04 Step 4) with platform token or OAuth client
M2S context	M2S-01 (inventory) and M2S-05 (execution waves) — this LAB slots into that sequence
Export utility	The provider binary itself, invoked directly; no separate download

> **Scope boundary.** Everything about *how Terraform works* lives in AUTOM. Everything about *what makes a Managed source different* lives here.

Learning Objectives

By the end of this LAB you will be able to:

- Decide whether Terraform, the SaaS Upgrade Assistant, or a combination fits your migration, and state the rule that keeps them from corrupting each other
- Choose between **bulk** and **iterative** export modes, and recognize that a prior Assistant run puts you in iterative mode whether you planned for it or not
- Run the export utility against a **Managed** cluster with the correct URL form, token type, and scopes
- Explain why redirecting OneAgents without `oneagentctl` can silently destroy the configuration you just migrated
- Sequence Terraform apply waves so that entity-ID-validating endpoints succeed
- Identify the configuration classes that no tool migrates, and handle them deliberately rather than discovering them at cutover

1. When Terraform Is the Right M2S Path

Dynatrace's own Managed upgrade documentation sanctions **three** approaches to configuration migration. Terraform is not an off-book route:

Approach	Dynatrace's framing	Best fit
SaaS Upgrade Assistant	Recommended default — "lets you easily import your Dynatrace Managed configuration to your SaaS environment and edit it in simple UI"	Teams without existing IaC; fastest path to a working tenant
Monaco	Repoint an existing Monaco deployment at the SaaS target	Teams already running Monaco against Managed
Terraform	Follow the dedicated migration guide	Teams already running Terraform, or who need the target tenant reproducible from source control

Choose Terraform when

Signal	Why it points to Terraform
You already manage Dynatrace config as Terraform against Managed	The migration is a provider re-target, not a new toolchain

Signal	Why it points to Terraform
The target tenant must be reproducible from source control	The Assistant produces a configured tenant, not a versioned definition of one
You are consolidating several Managed clusters into one SaaS tenant	Iterative export with duplicate handling is built for exactly this
Config changes need review before they land	`terraform plan` is a reviewable diff; the Assistant's UI is not
The same config must be applied to more than one target	Modules and workspaces; see AUTOM-09 §6

Choose the Assistant when

Signal	Why
No existing IaC practice	Terraform's value is in the ongoing lifecycle, not the one-time copy
A single Managed environment moving to a single fresh tenant	The Assistant does this in a guided flow with progress tracking and deploy results
The migration window is tight and the team is small	Lower ceremony, no state to manage

The one-writer rule

M2S-99 carries a Critical-priority best practice: *pick one primary tool, never mix approaches.* That rule is easy to misread as "Terraform is not allowed." It is not. The hazard it guards against is **two writers owning the same configuration object at the same time**.

Terraform maintains state — a record asserting that it owns specific objects. If the Assistant creates a second copy of an object Terraform already tracks, the next `terraform plan` sees drift it did not cause, and the apply that follows may recreate, orphan, or overwrite. The failure is not immediate; it surfaces one or two applies later, which is what makes it expensive.

> **The rule, stated precisely:** one writer per configuration schema, per migration window. Sequencing two tools is safe when ownership transfers cleanly and the

> handoff is explicit. Running them concurrently against the same schema is not.

Section 2 covers what a clean handoff actually requires.

2. Bulk vs Iterative — and Why the Assistant Forces Iterative

The export utility supports two migration strategies. Choosing the wrong one is the most common way a Terraform-based migration goes wrong on the first apply.

Strategy	Target precondition	Command shape	Dependency handling
Bulk	Target has **no existing configuration**	`-export -migrate`	Resource dependencies plus hardcoded entity IDs
Iterative	Target **already has configuration**	`-export -migrate -datasources <resourcename>`	Dependencies resolved via **data sources** plus hardcoded entity IDs

The decision that catches teams out

Bulk migration requires a genuinely empty target. In an M2S migration, a freshly provisioned SaaS tenant usually qualifies — which is why bulk is the normal case.

But a tenant stops being empty the moment anything else writes to it. In particular:

> **If you have already run the SaaS Upgrade Assistant against the target tenant — even a partial or exploratory run — you are in iterative mode.** Bulk export against that tenant will attempt to create objects that already exist.

The same applies if you are consolidating a second Managed cluster into a tenant that already received the first, or if platform defaults and onboarding config were applied during Step 4.

Duplicate handling (iterative only)

Iterative mode requires an explicit decision about what happens when an exported object collides with one already in the target:

Environment variable	Behavior
`DYNATRACE_DUPLICATE_REJECT=ALL`	Refuse to overwrite existing objects
`DYNATRACE_DUPLICATE_HIJACK=ALL`	Allow existing objects to be taken over and overwritten

Neither is a safe default for every case. `REJECT` preserves whatever the Assistant already created and is the conservative choice when the Assistant's output is authoritative. `HIJACK` makes Terraform the authority and is correct when you intend Terraform to own the object going forward — which is exactly the handoff described in section 1.

> **Handoff pattern.** Assistant migrates the bulk config during the cutover window; afterwards, a single iterative export with `DYNATRACE_DUPLICATE_HIJACK=ALL` brings those objects under Terraform state. From that point Terraform is the only writer. This satisfies the one-writer rule because ownership transfers at a defined moment rather than being contested continuously.

Discovering what is excluded

Some resources are excluded from export by default — notably `dynatrace_json_dashboard`, which is not exported unless the resource is named explicitly. Enumerate the current exclusion set before you plan the migration rather than discovering the gap after cutover:

```
# List every resource the export utility excludes by default
./terraform-provider-dynatrace -export -list-exclusions
```

3. What Exports from Managed, and What Never Will

Two distinct categories of gap exist, and conflating them causes bad planning. Some things the export utility cannot retrieve because the API will not expose them. Other things have no Managed equivalent at all, because they are Gen3 platform capabilities that only exist on the SaaS side.

3.1 Exports cleanly from a Managed source

Configuration domain	Notes
Settings 2.0 objects	Any schema present on the Managed version
Management zones	See the caveat below
Automatically applied tags	Rule-based tagging
Alerting profiles	
Network zones	`dynatrace_network_zone`; recreate before ActiveGate assignment (M2S-04 §6)
Calculated service metrics	May need rework — see section 6
Request attributes and naming rules	
SLOs	Requires a classic API token
Synthetic monitors	Requires a classic API token; private locations must exist in the target first
Dashboards	**Excluded by default** — must be named explicitly on the command line

> **Management zone caveat.** Management zones export faithfully, which is precisely the risk. On Gen3 SaaS the target model is segments plus IAM policies, and a literal port carries a model you are trying to retire straight into the new tenant. Decide before exporting whether each management zone is being migrated or replaced — the MZ2POL series covers the replacement path.

3.2 Never exports — the API will not return it

Item	Reason
Credential Vault contents	Secret values are write-only; the vault object may export, the secret never does
Access tokens and personal access tokens	Token values are never retrievable after creation
Extensions 1.0 and their endpoint credentials	Binary artifacts plus embedded secrets
Cloud integration credentials (AWS, Azure, GCP, Cloud Foundry, Kubernetes/OpenShift, Heroku, VMware)	Credential material

The export utility flags what it could not handle rather than failing silently — see section 4.3.

3.3 Requires manual migration — no tool moves these

Dynatrace documents an explicit list of settings that cannot be migrated automatically by **any** approach, Terraform included. Budget for it in Step 2 rather than discovering it in Step 5:

Category	Items
Identity and access	Account management — users, groups, permissions, IAM policies
Notifications	Problem notifications (Jira, OpsGenie, PagerDuty, Trello, VictorOps, xMatters); push notifications via the Dynatrace mobile app
Application observability	Metrics not linked to instances; rage-click configuration; JavaScript error configuration; mobile symbolication; request naming order; merged services; key requests and their references; server-side deep-monitoring exclusions (noisy exceptions, URLs, SQL bind variables)
Saved analysis	Multi-dimensional analysis saved views; Data Explorer saved queries
Service detection	Custom service rules order

Category	Items
Entity management	Remote environments; custom devices; manually applied entity tags; custom entity names and descriptions, including process groups

> **Blocking pre-migration task.** If the Managed environment uses simple detection rules, advanced detection rules, or declarative process-grouping rules, these must be migrated to **Process Grouping Rules before** the SaaS upgrade. This is not a post-cutover cleanup item — it gates the migration.

3.4 No Managed source exists

Grail buckets, segments, OpenPipeline pipelines, Workflows, and Gen3 documents (dashboards and notebooks as documents) have no counterpart on Managed. These are **net-new authorship** on the SaaS side, not migration. They belong to M2S-07 (Expand) and are the reason the M2S journey does not end at cutover.

Bucket design in particular must be settled **before first ingest** (M2S-04 §5) — retention and routing decisions are difficult to unwind once data has landed. ORGNZ covers the design.

4. Exporting from the Managed Cluster

4.1 Authentication — Managed differs from SaaS

This is the single most common setup error when teams reuse a SaaS Terraform configuration against a Managed source.

	Managed source	SaaS target
Environment URL form	`https://{your-managed-host}/e/{environment-id}`	`https://{your-tenant}.live.dynatrace.com`
Token type	Classic environment API token	Platform token or OAuth client (plus a classic API token for synthetics and SLOs)
Gen3 platform resources	Not present	Present

The export utility reads credentials from environment variables. Supply them that way rather than as command-line arguments — shell arguments appear in process listings:

```
# Source: the Managed environment being migrated FROM
export DYNATRACE_ENV_URL="https://{your-managed-host}/e/{environment-id}"
export DYNATRACE_API_TOKEN="$MANAGED_API_TOKEN"   # from a secret manager,
never inline

# Optional — output directory; defaults to ./configuration
export DYNATRACE_TARGET_FOLDER="./m2s-export"
```

4.2 Required API token scopes

Create the source token with exactly these scopes:

Scope	Covers
`settings.read`, `settings.write`	Settings 2.0 objects
`ReadConfig`, `WriteConfig`	Classic configuration API
`ExternalSyntheticIntegration`	Synthetic monitors
`CaptureRequestData`	Request attributes and data privacy
`credentialVault.read`, `credentialVault.write`	Credential vault objects (metadata only — not secret values)
`networkZones.read`, `networkZones.write`	Network zones

> Export is a read operation, but the utility requires the paired write scopes. Grant them on a token that is created for the migration and revoked when it completes.

4.3 Running the export

```
# Bulk — fresh SaaS target with no existing configuration
./terraform-provider-dynatrace -export -migrate

# Iterative — target already has configuration (e.g. the Assistant ran first)
export DYNATRACE_DUPLICATE_HIJACK=ALL
./terraform-provider-dynatrace -export -migrate -datasources
&lt;resourcename&gt;

# Dashboards are excluded by default — name them explicitly to include them
./terraform-provider-dynatrace -export -migrate dynatrace_json_dashboard
```

Useful flags beyond `-migrate`:

Flag	Effect	When to use it in M2S
`-migrate`	Dependencies plus hardcoded entity IDs	The migration default — prefer over `-ref` for cross-environment moves
`-ref`	Include data sources and dependencies	Same-environment export; see AUTOM-04
`-id`	Emit source object IDs as comments in the `.tf` files	**Strongly recommended** — your only mapping from Managed object to SaaS object after the fact
`-import-state`	Initialize modules and import into state	Adopting objects that already exist in the target
`-exclude`	Exclude named resources	Dropping management zones you intend to replace with segments
`-list-exclusions`	Print the default exclusion set	Run before planning
`-flat`	No module structure	Rarely wanted; AUTOM-09 §5 covers module strategy

4.4 Read the two diagnostic directories

The utility does not fail silently on configuration it cannot cleanly translate. It quarantines it:

Directory	Meaning	Action
`.flawed`	The configuration is invalid or could not be converted; each file carries an explanatory comment	Triage individually — often reveals config that was already broken on Managed
`.requires_attention`	Converted, but needs a human decision before apply	Review every file; do not bulk-apply

> **Do not skip this.** These two directories are the export utility's inventory of everything the migration will not do for you. Reviewing them is a Step 1 (Discover) deliverable, not a Step 5 surprise. Their contents feed directly into the manual work catalogued in section 3.3.

5. Preserving Entity IDs — the Configuration-Loss Trap

This section is the reason a Terraform M2S migration can appear to succeed and then quietly lose configuration days later.

The mechanism

`-migrate` writes **hardcoded entity IDs** into the generated HCL. Configurations that reference specific hosts, services, or process groups — dashboards pinned to an entity, alerting profiles scoped to one service, management zone rules naming an entity — carry those IDs across to the target.

Dynatrace's migration guidance is explicit about what breaks this:

> "Ensure OneAgent communication is reconfigured via `oneagentctl` for migration. Direct OneAgent migration might generate new entity IDs, potentially causing configuration loss."

If a host arrives in the SaaS tenant under a **new** entity ID, every exported configuration that referenced its old ID now points at nothing. The configuration applies without error — it simply refers to an entity that does not exist.

The consequence for sequencing

This couples two things that look independent on a project plan: **how you redirect OneAgents** and **whether your Terraform configuration survives**.

Redirect method	Entity IDs	Effect on exported config
Reconfigure in place via `oneagentctl`	Preserved	Exported entity references resolve
Uninstall and reinstall the agent	New IDs generated	Entity-scoped configuration silently orphaned

M2S-05 §4 documents the reconfigure method and flags it as the recommended path — that recommendation carries additional weight when Terraform is the migration tool, because the exported HCL has the old IDs baked in.

> **Practical rule.** If any part of your estate must be migrated by reinstall rather than reconfigure, treat entity-scoped configuration for those hosts as manual-rebuild work. Do not assume the exported HCL covers it.

Entity-ID-validating endpoints

A second, gentler failure mode: some API endpoints validate that referenced entity IDs actually exist at apply time, and reject the configuration if they do not.

This is not a defect — it is an ordering constraint. The fix is to apply again once the entities are present:

> Some API endpoints validate entity ID existence; re-apply after the entities are present in the target.

In practice this means the Terraform apply for entity-dependent configuration must come **after** OneAgents are reporting to SaaS and entities have been discovered — which is why section 7 places it in Step 6 rather than Step 5.

6. Applying to the SaaS Target

6.1 Re-point the provider

Export and apply are separate operations against separate environments. Swap the environment variables — and note the token model changes with the platform generation:

```
# Target: the SaaS tenant being migrated TO
export DYNATRACE_ENV_URL="https://{your-tenant}.live.dynatrace.com"
export DYNATRACE_PLATFORM_TOKEN="$SAAS_PLATFORM_TOKEN"
export DYNATRACE_HTTP_OAUTH_PREFERENCE=true

# Synthetics and SLOs still require a classic API token alongside the platform token
export DYNATRACE_API_TOKEN="$SAAS_API_TOKEN"
```

> Provider version discipline matters more here than in steady-state work, because you are applying config generated by one binary using another. Pin the version explicitly and use the same provider version for export and apply. AUTOM-04 §3 carries the current authentication and version guidance; the corpus last verified provider behavior at **v1.100.0 (07/08/2026)**.

6.2 Apply in the M2S wave order

Do not apply the whole export in one transaction. M2S-05 §2 establishes a dependency-ordered wave sequence; Terraform should follow the same order, because the dependencies are properties of Dynatrace, not of the tool:

Wave	Contents	Terraform scoping
1 — Foundation	Network zones, management zones or segments, tagging rules	`-target` the foundation module
2 — Instrumentation	Process groups, service detection, request attributes	Apply after Wave 1 succeeds
3 — Monitoring	Alerting profiles, anomaly detection, SLOs	Depends on Wave 2 naming

Wave	Contents	Terraform scoping
4 — Visualization	Dashboards, saved views	Last — most entity-dependent

```
# Wave-by-wave apply, reviewing the plan at each boundary
terraform plan  -target=module.foundation -out=wave1.tfplan
terraform apply wave1.tfplan
```

6.3 Triage the first plan

The first `terraform plan` against the target is a diagnostic, not a formality. Expect three categories:

Plan output	Meaning	Action
Create	Object does not exist in target	Normal for bulk migration
Update in place	Object exists and differs	Expected in iterative mode; confirm `HIJACK` was intended
Replace / destroy	Terraform intends to remove something	**Stop.** Nothing in a migration should require destroying target config

6.4 Classic endpoints may reject configuration that was valid on Managed

Some classic API endpoints have tightened their validation since the Managed configuration was authored. The documented example: calculated service metrics now require either a management zone or service property conditions.

The export is faithful to the source; the target's validation is stricter. This is normal and it surfaces at apply. Fix forward — do not weaken the target to accept legacy config.

6.5 Re-apply for entity-dependent configuration

Per section 5, configuration referencing entity IDs must be applied after those entities exist in the target. The sequence is:

1. Apply Waves 1–3 (configuration that does not depend on discovered entities)

2. Redirect OneAgents via `oneagentctl` (M2S-05 §4) and wait for entity discovery
3. Re-run `terraform apply` — endpoints that previously rejected entity references now accept them
4. Apply Wave 4

7. Where Terraform Actions Land in the Nine Steps

This LAB is an appendix, not a tenth step. Every action above belongs to an existing step:

M2S Step	Terraform action	Notebook
1 — Discover	Run `-list-exclusions`; run a trial export; review `.flawed` and `.requires_attention` to size the manual remainder	M2S-01
2 — Strategize	Choose Terraform as the primary tool, or define the Assistant→Terraform handoff; budget the manual list from section 3.3	M2S-02
3 — Design	Decide management zones vs segments before export; design the target repo layout and state backend	M2S-03, AUTOM-09
4 — Prepare	Provision the target tenant and tokens; create network zones; **migrate detection rules to Process Grouping Rules**; settle Grail bucket design before first ingest	M2S-04
5 — Execute	Export from Managed; apply Waves 1–3; redirect OneAgents via `oneagentctl`	M2S-05
6 — Integrate	Re-apply for entity-dependent config; apply Wave 4; rebuild notification channels and cloud credentials by hand	M2S-06
7 — Enable	Hand the repo to the teams who will own it; document the promotion flow	M2S-07, AUTOM-09 §10
8 — Expand	Author Gen3-only resources — Grail buckets, segments, OpenPipeline, Workflows	M2S-08, ORGNZ
9 — Optimize	Establish drift detection; decommission the Managed cluster and revoke its export token	M2S-09

> **The trial export in Step 1 is the highest-value item in this table.** It converts "we think Terraform will handle our config" into a reviewed list of what it will not. Running it costs an afternoon and reshapes the Step 2 plan.

8. The Manual Remainder

M2S-02 states the 90/10 rule: roughly 90% of configuration migrates automatically, and the remaining 10% consumes 90% of the manual effort. Terraform does not change that ratio — it changes *where the 10% shows up*, moving it from the Assistant's deploy-results screen into your repository.

Handle each class deliberately:

Class	Terraform-side handling
Credential Vault secrets	Declare the vault object in HCL; inject values from your secret manager at apply time. Never commit values, and note that they land in state — see below
API tokens	Mint out-of-band. The provider has no Platform Token resource, and `dynatrace_api_token` writes the token value into state in plaintext
Cloud integration credentials	Recreate per provider. CLOUD series covers the target-side setup
Extensions 1.0	Re-upload; binary artifacts are outside Terraform's reach
Problem notifications	Rebuild per channel. On Gen3, prefer Workflows — WFLOW covers the target pattern; ALERT-03 covers routing and cost
Account management / IAM	Not a migration. Managed cluster groups do not map to SaaS account IAM. Author fresh with `dynatrace_iam_*` — AUTOM-95 and IAM-95 are the LABs
Manually applied tags, custom device names, custom entity names	Manual re-application, or convert to rule-based tagging so it becomes Terraform-manageable going forward
Saved views and Data Explorer queries	Rebuild as Notebooks or Dashboards on the target — DASH covers the Gen3 model

> **State is now part of your security perimeter.** Injecting credential-vault values or minting tokens through Terraform places sensitive material in state in plaintext, regardless of `sensitive = true`. Encrypted remote backends and least-privilege state access become mandatory rather than advisable. AUTOM-04 and AUTOM-09 §3/§8 cover backend hardening in full.

Turning the remainder into an asset

The manual items are the ones the Assistant would also leave behind. The difference is that once you have rebuilt them in HCL, they are versioned and reproducible — so the second environment, the DR tenant, and next year's audit all cost dramatically less. Migrating with Terraform is more work at cutover and less work permanently afterwards.

9. Post-Cutover: Steady-State Ownership

This section applies to **every** M2S migration, including those that used the Assistant end to end. A migrated tenant that nobody manages as code drifts immediately.

Adopting an Assistant-migrated tenant

If the Assistant performed the migration, the target now holds configuration that no code describes. Bring it under management with an export of the **target**:

```
# Point at the SaaS target and export what is actually there
export DYNATRACE_ENV_URL="https://{your-tenant}.live.dynatrace.com"
export DYNATRACE_PLATFORM_TOKEN="$SAAS_PLATFORM_TOKEN"
export DYNATRACE_HTTP_OAUTH_PREFERENCE=true

# -import-state adopts existing objects into Terraform state
./terraform-provider-dynatrace -export -ref -id -import-state
```

This is the clean handoff from section 1: the Assistant owned the cutover, Terraform owns everything after it, and the transfer happens at one identifiable moment.

Establish a drift baseline

Once state reflects reality, a non-zero plan means someone changed configuration in the UI:

```
# Exit codes: 0 = no drift, 1 = error, 2 = drift detected
terraform plan -detailed-exitcode
```

Run this on a schedule. In the weeks after a migration it is the fastest way to detect configuration being hand-patched during incident response — which is normal, and which needs to be folded back into code before it is forgotten.

10. Validation

Terraform confirms that configuration was *written*. These queries confirm it is *taking effect* on the target. Run them after each apply wave.

10.1 Host arrival parity

Compare against the source-cluster host count from M2S-01. A shortfall means agents have not finished redirecting — investigate before applying entity-dependent waves.

```
// Host count reporting to the SaaS target.
// Compare against the Managed inventory captured in M2S-01.
smartscapeNodes "HOST"
| summarize hosts = count()
```

10.2 Host-group coverage

Host groups drive naming, tagging, and management-zone or segment rules. A large `null` bucket means hosts arrived without host-group assignment, and every downstream rule keyed on host group will under-match.

```
// Host-group distribution on the target.
// A large null group means host-group assignment was lost during redirect.
smartscapeNodes "HOST"
| summarize hosts = count(), by:{dt.host_group.id}
| sort hosts desc
```

10.3 Tag-rule effectiveness

Applying an auto-tag rule is not the same as that rule matching anything. This checks that a Terraform-managed tagging rule actually produces tags on the target — replace `app.owner` with a tag key your rules assign.

```
// Do Terraform-managed tagging rules actually match hosts on the target?
// Replace app.owner with a tag key your ruleset assigns.
smartscapeNodes "HOST"
| fieldsAdd tags = getNodeField(id, "tags")
| fieldsAdd hasOwner = isNotNull(tags[app.owner])
| summarize tagged = countIf(hasOwner == true), total = count()
```

> **Interpretation.** `tagged` well below `total` usually means the rule's conditions reference entity properties that differ on SaaS, or that it depends on manually applied tags — which section 3.3 lists as non-migrating. This query is the fastest way to find rules that migrated syntactically but stopped working semantically.

M2S-05 §5 carries the broader post-wave validation set — service discovery, trace flow, log ingestion, and metric gap checks. Run those alongside these three.

11. Rollback and Safety

Terraform introduces failure modes the Assistant does not have. All of them are avoidable.

Risk	Control
`terraform destroy` run against a live tenant	Never available in a migration pipeline. Restrict the apply role; keep destroy out of CI entirely
An apply that plans replacements	Treat any `replace` or `destroy` line in a migration plan as a stop condition (section 6.3)
State loss mid-migration	Remote backend with versioning and at-rest encryption **before** the first apply — AUTOM-09 §3
State and reality diverging	Back up state before each wave; `terraform plan` between waves, never blind applies

Risk	Control
Accidental deletion of foundation objects	`lifecycle { prevent_destroy = true }` on network zones, segments, and management zones
Export token outliving the migration	Revoke the Managed API token at decommission (Step 9)

The actual rollback path

Terraform is not the rollback mechanism. During migration the **Managed cluster remains authoritative** until validation passes — that is the rollback, and it is the same rollback M2S-04 §8 defines for every migration approach regardless of tooling.

Rolling back Terraform means reverting the repository to the previous commit and re-applying. That restores *configuration*, not *data*, and it does not return OneAgents to the Managed cluster. Agent rollback is `oneagentctl` re-pointing, documented in M2S-04 §8.

> Keep the Managed cluster running through the parallel-operation window. M2S-02 §7 treats that window as non-negotiable — Davis needs two to four weeks in SaaS to build trustworthy baselines. Terraform does not shorten it.

12. Completion Checklist

Discover and plan

- [ ] `-list-exclusions` run; default exclusion set reviewed
- [ ] Trial export completed against the Managed source
- [ ] `.flawed` and `.requires_attention` reviewed; manual remainder sized
- [ ] Section 3.3 manual list walked; owners assigned per item
- [ ] Detection rules migrated to Process Grouping Rules **before** upgrade
- [ ] Bulk vs iterative decided; if the Assistant will run first, duplicate handling chosen

Prepare

- [ ] Source token created with the section 4.2 scopes; stored in a secret manager
- [ ] Target tenant provisioned; platform token and classic API token both available
- [ ] Remote state backend with versioning and encryption in place

- [ ] Provider version pinned identically for export and apply
- [ ] Management zone vs segment decision made per zone
- [ ] Grail bucket design settled before first ingest

Execute

- [ ] Export run with `-migrate` and `-id`
- [ ] Generated HCL committed and reviewed before any apply
- [ ] Waves 1–3 applied in order, plan reviewed at each boundary
- [ ] No `replace` or `destroy` lines accepted in any migration plan
- [ ] OneAgents redirected via `oneagentctl` — not uninstall/reinstall
- [ ] Re-apply completed after entity discovery; Wave 4 applied

Validate and hand off

- [ ] Host parity, host-group coverage, and tag-effectiveness queries run
- [ ] M2S-05 §5 post-wave validation completed
- [ ] `prevent_destroy` set on foundation resources
- [ ] Drift detection scheduled with `-detailed-exitcode`
- [ ] Repo ownership handed to the operating teams
- [ ] Managed API token revoked at decommission

Summary

Takeaway	Detail
Terraform is a sanctioned M2S path	Dynatrace's Managed upgrade docs name Assistant, Monaco, and Terraform. The Assistant is recommended; Terraform is legitimate
"Never mix" means one writer per schema	Sequencing tools with an explicit ownership handoff is safe; concurrent writes are not
A prior Assistant run forces iterative mode	Bulk export requires a genuinely empty target; plan duplicate handling deliberately
`-migrate` hardcodes entity IDs	Which makes `oneagentctl` reconfiguration a correctness requirement, not a preference

Takeaway	Detail
Redirect method determines config survival	Uninstall/reinstall generates new entity IDs and silently orphans entity-scoped configuration
Entity-dependent config applies twice	Once before discovery, again after — endpoints validate entity existence
Read `.flawed` and `.requires_attention`	They are the export utility's inventory of what the migration will not do for you
The manual remainder is unchanged	Terraform relocates the 10%, it does not shrink it — but it makes the rebuild permanent
Every migration needs section 9	Even an Assistant-only migration should end with `-import-state` adoption and drift detection

Next Steps

- **M2S-05** — the execution waves and `oneagentctl` redirect procedure this LAB depends on
- **M2S-06** — reconnecting the integrations that do not migrate
- **AUTOM-04 / AUTOM-09** — provider configuration, state backends, module strategy, GitOps promotion
- **AUTOM-95 / IAM-95** — authoring SaaS account IAM in HCL, which is net-new work
- **MZ2POL** — replacing management zones with policies and segments instead of porting them
- **ORGNZ** — Grail bucket and segment design for the Expand phase

References

- Export utility, Managed (DT docs) — flag reference, required scopes, `.flawed` / `.requires_attention` behavior
- Migrate between different environments using Terraform (DT docs) — bulk vs iterative, duplicate handling, `oneagentctl` entity-ID guidance
- Settings that require manual migration (DT docs) — the authoritative non-migrating list and the Process Grouping Rules prerequisite
- terraform-provider-dynatrace (Dynatrace GitHub) — provider source and export implementation

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.